

合肥工业大学

课程设计

设计题目 计算机图形学课程设计

学生姓名 张致远

学 号 2023213980

专业班级 信息与计算科学 23-1 班

指导教师 霍星

2026 年 5 月 24 日

合肥工业大学课程设计任务书

设计 题目	计算机图形学课程设计	成绩	
课 程 设 计 主 要 内 容	完成了所有四个题目，并用 python+pygame 库设计了交互式的 UI 来呈现题目一：星空绘制，题目二：鸭子，题目三：游泳的鸭子，题目四：松鼠松果，并在这个过程中总结了收获基本绘制直线，圆，填充等算法没有调包，均为基础手写完成。		
指 导 教 师 评 语	建议：从学生的工作态度、工作量、设计（论文）的创造性、学术性、实用性及书面表达能力等方面给出评价。 签名：2025 年 月 日		

目 录

一、设计题目与要求	1
1.1 设计题目	
1.2 设计内容	
1.3 设计目标	
二、系统功能介绍	2
三、方案设计	4
3.1 总体方案设计	
四、详细设计	6
4.1 题目一：星空图绘制	
4.2 题目二：鸭子绘制与变换	
4.3 题目三：游泳的鸭子	
4.4 题目四：松树与松果	
五、总结	34
5.1 程序结构优缺点分析	
5.2 出错原因分析	

一、设计题目与要求

1.1 设计题目

基于 Python 与 Pygame 的计算机图形学综合作品——星空图、鸭子动画与松树场景绘制系统。

1.2 设计内容

本作品按合肥工业大学《计算机图形学》课程设计任务书的要求，使用 Python 与 Pygame 实现以下四个子模块：

- (1) **题目一（必做）** 利用点、线、圆、多边形等计算机基本图元绘制一幅星空图；
- (2) **题目二** 绘制一只鸭子，并对鸭子进行旋转、平移等几何变换；
- (3) **题目三（提高题）** 绘制一只游泳的鸭子；
- (4) **题目四（难题）** 绘制一棵枝叶摇动的松树和掉落的松果。

本报告依次给出四个子模块的方案设计、详细设计与实现，其中题目一已完整实现，其余子模块的实现随开发推进逐步补充。整套作品强调以下两点：

- **底层算法自实现：**直线、圆、多边形填充等基本图元的生成 不直接调用 Pygame 提供的高层 API（如 `pygame.draw.line`），而是亲自实现 Bresenham 直线生成、中点画圆、扫描线多边形填充等经典算法，通过 `Surface.set_at()` 直接写像素，体现对图形学底层原理的掌握。
- **统一交互框架：**所有子模块共享同一套自定义 UI 组件（按钮、开关、滑块），具备菜单栏 / 工具栏 / 参数面板 / 主画布的统一布局，符合任务书“界面美观、衔接自然、注重交互性”的要求。

1.3 设计目标

- (1) 巩固和综合应用所学的计算机图形学理论知识，培养学生理论联系实际和独立思考问题的能力；
- (2) 掌握 Bresenham 直线、中点画圆、扫描线多边形填充等基础图元生成算法的推导与编码实现；

- (3) 培养图形应用程序的工程化开发能力——模块划分、控件封装、性能优化与文档撰写；
- (4) 通过四道题目的递进设计，体会从静态图形 → 几何变换 → 动画 → 复杂交互动画的渐进过程，强化对二维图形系统的整体认识。

二、系统功能介绍

整套作品由四个独立子模块组成，每个子模块对应任务书中的一道题目，均以一个独立可执行 Python 脚本形式提供。所有子模块共享统一的界面骨架：**顶部工具栏 + 右侧参数控制面板 + 主画布**。

题目一：星空图绘制

启动 python 题目 1_ 星空图/main.py 进入星空场景，主要功能包括：

- **场景渲染**：渐变天空背景；满天可调密度的小星星；一颗会缓慢自转、带光芒的大五角星；一轮带月坑与光晕的月亮；沿对角线划过的彗星；锯齿状的山脉剪影地平线。
- **工具栏按钮**：重新生成（随机化星点与山脉）、切换主题（深夜 / 黄昏 / 极光三套配色）、保存图片（导出当前画布为 PNG）、关于（弹窗显示算法说明）。
- **工具栏开关**：月亮 / 彗星 / 动画的独立显示开关。
- **参数面板**：星星数量（10~250）、大星星尺寸（30~120 像素）两个滑块。
- **快捷键**：R 重新生成、T 切换主题、S 保存图片、Space 暂停/恢复动画、ESC 退出。

题目二：鸭子绘制与几何变换

启动 python 题目 2_ 鸭子/main.py 进入鸭子变换演示。鸭子由 **身体 / 腹部 / 翅膀 / 尾巴 / 头 / 嘴 / 眼 / 腮红** 等 10 个部件组成，全部由基本图元（椭圆多边形、圆、菱形多边形）拼合，所有变换通过自实现的 2D 齐次坐标矩阵完成。主要功能包括：

- **4 个滑块**：旋转角度（ $-180^\circ \sim 180^\circ$ ）、缩放比例（0.5 ~ 2.5）、平移 X / Y。
- **鼠标拖拽**：在画布上直接拖动鸭子（滑块自动跟随同步）。
- **键盘热键**：方向键 = 平移、Q/E = 旋转、+/- = 缩放、R = 重置、Space = 切换动画。

- **圆周动画：**鸭子沿半径 180 的圆周持续运动，且始终面向切线方向——直接可视化 $T(\theta) \cdot R(\theta)$ 的矩阵复合效果。
- **可视化辅助：**背景棋盘网格；鸭子本地坐标系的红色 X 轴和绿色 Y 轴（随变换实时偏转）；右侧面板实时显示当前 3×3 变换矩阵；可选的运动轨迹绘制。

题目三：游泳的鸭子（提高题）

启动 python 题目 3_ 游泳的鸭子/main.py 进入游泳场景。在题目二的鸭子和变换框架基础上加入以下新内容：

- **水面环境：**天空 + 水面的双段渐变背景、远山剪影、太阳/月亮、正弦扰动的动态水面线。
- **游泳动画：**鸭子水平往返游动，到边界自动掉头并镜像翻转朝向；叠加正弦的上下浮沉与左右摇摆，由统一的复合变换矩阵 $M = T \cdot R \cdot S$ 实现。
- **Sutherland–Hodgman 多边形裁剪**（本题核心新算法）：把鸭子身体与腹部多边形按水线裁剪，水线下方的子多边形重新填充为水蓝色，可视化”水下染色”。开启”裁剪调试”模式可看到水线（红色直线）与裁剪后子多边形的描边（黄色）以及裁剪交点（红点）。
- **涟漪：**鸭子每游一段距离从尾后向水面发射圆形涟漪，用中点画圆生成圆周像素，只保留水面附近的部分实现”扁平水波”视效，随帧数渐隐。
- **交互：**游泳速度 / 浮沉幅度 / 摇摆幅度 / 水位 4 个滑块；涟漪 / 裁剪 / 裁剪调试 3 个开关；重置 / 暂停 / 反向 / 切换主题 / 保存图片 / 关于 6 个按钮；鼠标可拖动鸭子（拖动时自动暂停 auto-swim）；Space 暂停、R 重置、ESC 退出。
- **三套主题：**白昼 / 黄昏 / 月夜。

题目四：枝叶摇动的松树与掉落的松果（难题）

启动 python 题目 4_ 松树松果/main.py 进入森林场景。本模块综合运用了前三题的全部算法（Bresenham、中点画圆、扫描线填充、2D 齐次变换、Sutherland–Hodgman 裁剪虽未直接使用但保留在算法库），新引入 De Casteljau Bezier 曲线和基于矩阵的层级化变换，并叠加一套简单的刚体物理仿真。主要功能：

- **松树：**一根中央树干 + 5 层枝叶 tier 自下而上堆叠。每层 tier 的下垂底边由二次 *Bezier* 曲线采样得到，侧边由斜直线收拢到顶点，组合为一个 20 顶点多边形并扫描线填充。
- **风力摇摆：**每层 tier 围绕自身底部独立做正弦旋转，摇摆幅度随高度递增（顶层最大）；所有 tier 的旋转矩阵叠加到树干基变换之上构成 层级化变换链，与骨骼动画思想相同。
- **松果三状态机：**ATTACHED（挂着，每帧同步当前枝头世界位置）→ FALLING（自由落体，受重力 + 风 + 阻力 + 自旋）→ LANDED（落地停在草坪）。
- **自动掉落 + 手动摇晃：**每 150~280 帧随机摇下一颗；”摇晃”按钮一次摇下约 1/3 的挂着松果；鼠标点击画布距某颗挂果 80px 内可定点摇下它。
- **算法可视化：**开启”Bezier 控制点显示”开关后，底层 tier 的 3 个控制点（红点）与控制多边形（粉色折线）会实时绘制在画布上，直观显示曲线的几何意义。
- **交互：**风力 / 摇摆速度 / 松果数量 3 个滑块；摇晃 / 全部掉落 / 重置 / 暂停 / 切换主题 / 保存图片 / 关于 7 个按钮；S 摇晃、R 重置、Space 暂停、ESC 退出。
- **三套主题：**春日 / 黄昏 / 雪夜（雪夜额外有受风力影响的飘雪花粒子）。

三、方案设计

3.1 总体方案设计

开发环境

项目	选用	说明
编程语言	Python 3.11	跨平台、表达力强，社区图形与数学库丰富
图形库	Pygame 2.6	基于 SDL2，提供 <code>Surface</code> 像素级操作能力，便于自实现底层算法
操作系统	Windows 11	开发与运行环境
排版工具	XeLaTeX (TeX Live 2025)	撰写本课程设计报告

Table 1: 开发环境与依赖

工程结构 以” 算法层 / 控件层 / 应用层” 三层结构组织代码。每道题目放在独立的子目录下，共用统一的算法模块与控件模块：

```

1  图形学课设/
2      课程设计报告.tex
3      题目1_星空图/
4          algorithms.py # 底层图元生成算法
5          widgets.py # 自定义 Pygame UI 控件
6          main.py # 应用入口、场景渲染、事件循环
7      题目2_鸭子/
8      题目3_游泳的鸭子/
9      题目4_松树松果/

```

设计原则

- **关注点分离**: `algorithms.py` 不依赖任何渲染目标，只输入几何参数、输出像素坐标列表；`main.py` 拿到像素列表后批量写入 `Surface`。如此算法可独立单元测试，也可移植到 Tkinter、PIL 等其他渲染后端。
- **即时模式 UI (immediate-mode)**: 自定义按钮 / 开关 / 滑块每帧重画一遍，状态保存在控件对象自身，逻辑简单清晰，易于扩展。
- **静态层缓存**: 背景渐变、山脉、月亮等帧间不变的内容预先渲染到一张缓存 `Surface` 上，每帧只需 `blit` 复用；逐帧只重绘动态元素（闪烁的小星星、旋转的大五角星、移

动的彗星)，显著降低 `set_at` 调用次数，使得 60FPS 渲染成为可能。

- **字体可移植：**通过文件路径直接加载 Windows 系统字体（如 `msyh.ttc`），避开 `pygame.font.SysFont` 在某些系统注册表上的崩溃问题。

四、详细设计

4.1 题目一：星空图绘制

4.1.1 算法原理

题目一要求”利用点、线、圆、多边形等计算机基本图元的绘制绘制一幅星星图”。本实现以亲自实现经典图元算法为核心，所有星点、圆、五角星、山脉均由下述算法生成像素，然后写入 Pygame 的 `Surface`，未使用 `pygame.draw` 任何高层 API。

（一）Bresenham 直线生成算法

设直线两端点为 (x_0, y_0) 与 (x_1, y_1) ，记 $\Delta x = |x_1 - x_0|$ 、 $\Delta y = |y_1 - y_0|$ 。光栅化的核心问题是：在整数像素网格上找到最接近理想直线 $y = kx + b$ 的像素序列。

Bresenham 算法通过维护一个误差累积量 `err`，只用加减法和比较完成判断，避免浮点除法。算法初值 $\text{err} = \Delta x - \Delta y$ ，每步迭代根据 $e_2 = 2\text{err}$ 与 $\pm\Delta x, \pm\Delta y$ 的比较结果，独立判断 x, y 是否前进一步。通过 $s_x = \pm 1, s_y = \pm 1$ 提前预处理方向，一份代码可统一处理八个斜率象限。

伪代码如下：

```
1 def bresenham_line(x0, y0, x1, y1):
2     dx, dy = abs(x1 - x0), abs(y1 - y0)
3     sx = 1 if x0 < x1 else -1
4     sy = 1 if y0 < y1 else -1
5     err = dx - dy
6     x, y = x0, y0
7     pts = []
8     while True:
9         pts.append((x, y))
10        if x == x1 and y == y1:
11            break
12        e2 = 2 * err
```

```

13     if e2 > -dy: err -= dy; x += sx
14     if e2 < dx: err += dx; y += sy
15 return pts

```

Listing 1: Bresenham 直线生成 (algorithms.py 摘录)

应用场景：小星星的十字光、大五角星的描边、12 束外发散光芒、彗星拖尾。

(二) 中点画圆算法 (八对称性)

圆方程 $F(x, y) = x^2 + y^2 - r^2$ 。圆内 $F < 0$ 、圆上 $F = 0$ 、圆外 $F > 0$ 。利用圆的八对称性，只需绘制 $45^\circ \sim 90^\circ$ 这一段弧 (满足 $x \leq y$ 的区段)，其余七段通过对称写入。

从 $(0, r)$ 出发，每次 x 自增 1，下一像素位置在 $(x+1, y)$ 与 $(x+1, y-1)$ 之间二选一。以中点 $M = (x+1, y-1/2)$ 代入 F ，记其值为 d (中点判别式)。推导可得递推关系：

$$d_0 = 1 - r, \quad d_{k+1} = \begin{cases} d_k + 2x + 3, & d_k < 0 \text{ (取右侧像素, } y \text{ 不变)} \\ d_k + 2(x - y) + 5, & d_k \geq 0 \text{ (取右下像素, } y \text{ 减 1)} \end{cases}$$

八对称点 $(\pm x, \pm y)$ 、 $(\pm y, \pm x)$ 共 8 个像素一次性写入：

```

1 def midpoint_circle(cx, cy, r):
2     pts = []
3     x, y, d = 0, r, 1 - r
4     while x <= y:
5         pts.extend([
6             (cx+x, cy+y), (cx-x, cy+y), (cx+x, cy-y), (cx-x, cy-y),
7             (cx+y, cy+x), (cx-y, cy+x), (cx+y, cy-x), (cx-y, cy-x),
8         ])
9         if d < 0:
10             d += 2 * x + 3
11         else:
12             d += 2 * (x - y) + 5
13             y -= 1
14             x += 1
15     return pts

```

Listing 2: 中点画圆 (algorithms.py 摘录)

在中点画圆基础上, 额外提供 `midpoint_circle_fill` 函数: 每一对镜像扫描线 $cy \pm y$ 与 $cy \pm x$ 之间填充水平段, 即可得到**实心圆**填充。

应用场景: 月亮、月坑、月晕、彗头光晕、稍大的小星星实心圆。

(三) 扫描线多边形填充算法

对凸或凹多边形, 依次用一组水平扫描线 $y = y_{\min}, \dots, y_{\max}$ 扫过。对每条扫描线:

- (1) 求扫描线与所有非水平边的交点集合;
- (2) 端点处理约定为**包含下端点** y_0 、**不含上端点** y_1 (即 $y_0 \leq y < y_1$), 避免顶点处出现奇数个交点;
- (3) 按线性插值计算交点 $x = x_0 + \frac{y - y_0}{y_1 - y_0}(x_1 - x_0)$;
- (4) 交点按 x 排序后, 两两配对 $[x_{2k}, x_{2k+1}]$, 在区间内逐像素填充。

```
1 def scanline_fill_polygon(vertices):
2     ys = [int(v[1]) for v in vertices]
3     ymin, ymax = min(ys), max(ys)
4     n = len(vertices)
5     pts = []
6     for y in range(ymin, ymax + 1):
7         xs = []
8         for i in range(n):
9             x0, y0 = vertices[i]
10            x1, y1 = vertices[(i + 1) % n]
11            if y0 == y1: # 水平边跳过
12                continue
13            if y0 > y1: # 保证 y0 < y1
14                x0, y0, x1, y1 = x1, y1, x0, y0
15            if y0 <= y < y1: # 端点约定 [y0, y1)
16                t = (y - y0) / (y1 - y0)
17                xs.append(x0 + t * (x1 - x0))
```

```

18     xs.sort()
19     for i in range(0, len(xs) - 1, 2):
20         for xx in range(round(xs[i]), round(xs[i+1]) + 1):
21             pts.append((xx, y))
22     return pts

```

Listing 3: 扫描线多边形填充 (algorithms.py 摘录)

应用场景：大五角星的实心填充（10 顶点凹多边形）、山脉剪影地形的填充（多个三角峰构成的不规则多边形）。

（四） n 角星顶点生成

正 n 角星由 $2n$ 个顶点构成，外/内两组顶点交替排列在外接圆 R 与内接圆 r 上。取角度均匀分布 $\theta_i = i \cdot \pi/n - \pi/2 + \alpha$ （ α 为整体旋转角，初始顶点指向正上方）：

$$P_i = (c_x + R_i \cos \theta_i, c_y + R_i \sin \theta_i), \quad R_i = \begin{cases} R, & i \text{ 偶数} \\ r, & i \text{ 奇数} \end{cases}$$

本实现中默认内/外半径比取 $r/R \approx 0.382$ （接近黄金分割比例 $1/\varphi^2 \approx 0.382$ ），五角星视觉上最接近标准比例。生成的 10 个顶点直接传入 `scanline_fill_polygon`，即得到完整的实心五角星。

4.1.2 主要功能模块设计

题目一以“算法层 / 控件层 / 应用层”三层结构组织，对应三个 Python 文件，模块依赖关系如图 1 所示。



Figure 1: 题目一模块依赖关系

- `algorithms.py` ——不依赖任何图形库，纯函数返回像素坐标列表。可作为通用模块

被其他三道题复用。

- `widgets.py` ——实现 `Button`（按钮）、`Toggle`（开关）、`Slider`（滑块）三类即时模式控件，仅依赖 `pygame.draw` 画底色与边框，文字使用从文件加载的中文字体渲染。
- `main.py` ——两个核心类：
 - `StarScene` ——场景对象，封装”参数 + 静态缓存层 + 帧渲染”。持有星点列表、山脉顶点、当前主题、动画计数器等状态。
 - `App` ——应用对象，封装”窗口 + UI 控件 + 主循环”。组装工具栏和侧边栏，分发事件，调用 `StarScene.render()`。

4.1.3 实现流程

画像素的统一接口 算法层返回 `[(x, y), ...]` 像素列表，渲染层用统一辅助函数 `_put` 写入 `Surface`，并做越界裁剪：

```
1 def _put(self, surf, points, color):
2     w, h = surf.get_size()
3     for x, y in points:
4         if 0 <= x < w and 0 <= y < h:
5             surf.set_at((x, y), color)
```

Listing 4: 像素写入辅助

大五角星的合成绘制 综合调用三大算法：

```
1 def _draw_big_star(self, target):
2     color = self.theme["big_star"]
3     cx, cy, r = self.canvas_w - 200, 150, self.big_star_size
4     rot = (self.frame * 0.004) if self.animate else 0.0
5
6     verts = star_vertices(cx, cy, r, num_points=5, rotation=rot)
7     self._put(target, scanline_fill_polygon(verts), color) # 扫描线填充
8
9     outline = tuple(min(255, c + 30) for c in color)
10    for i in range(len(verts)):
```

```

11     self._put(target,
12                 bresenham_line(*verts[i], *verts[(i+1) % len(verts)]),
13                 outline) # Bresenham 描边
14
15 self._put(target, midpoint_circle_fill(cx, cy, 4), (255, 255, 255)) # 中心高
    光
16
17 for ang_deg in range(0, 360, 30): # 12 束外发散光芒
18     ang = math.radians(ang_deg) + rot
19     x1 = cx + int((r + 10) * math.cos(ang))
20     y1 = cy + int((r + 10) * math.sin(ang))
21     x2 = cx + int((r + 30) * math.cos(ang))
22     y2 = cy + int((r + 30) * math.sin(ang))
23     self._put(target, bresenham_line(x1, y1, x2, y2),
24                 tuple(int(c * 0.65) for c in color))

```

Listing 5: 大五角星的渲染过程（main.py 摘录）

静态层缓存的实现 把”帧间不变”的元素先画到一张额外的 Surface 上，缓存它的签名（主题、月亮显示与否、山脉形状）；每帧渲染时如果签名未变，直接 blit 缓存层即可，避免重复执行高代价的 set_at。关键代码节选：

```

1 def render(self):
2     sig = (self.theme_name, self.show_moon, tuple(self.mountain_verts))
3     if self._cache_signature != sig or self._static_cache is None:
4         cache = pygame.Surface((self.canvas_w, self.canvas_h))
5         self._draw_gradient_bg(cache)
6         self._draw_mountains(cache)
7         if self.show_moon:
8             self._draw_moon(cache)
9         self._static_cache = cache
10        self._cache_signature = sig
11
12 self.surface.blit(self._static_cache, (0, 0)) # 一次 blit 复用静态层
13 self._draw_small_stars(self.surface) # 动态：闪烁

```

```

14     self._draw_big_star(self.surface) # 动态: 旋转
15     if self.show_comet:
16         self._draw_comet(self.surface) # 动态: 移动

```

Listing 6: 静态层缓存 (main.py 摘录)

4.1.4 运行效果



Figure 2: 题目一 • 星空图运行主界面 (深夜主题)

4.2 题目二：鸭子绘制与变换

题目二要求”绘制一只鸭子，并对鸭子进行旋转、平移等操作”。本模块以 二维齐次坐标变换为核心算法，把鸭子拆成若干部件分别定义在 局部坐标系中，渲染时统一施加一个 3×3 复合变换矩阵。

4.2.1 算法原理

(一) 为何引入齐次坐标

在普通笛卡尔坐标下，平移变换 $(x, y) \rightarrow (x + t_x, y + t_y)$ 只能用加法表示，不能写成矩阵乘法；而旋转、缩放是乘法形式。这种“非线性”使得变换序列难以统一表达。引入齐次坐标——把二维点 (x, y) 表示成三维向量 $(x, y, 1)^T$ 后，平移、旋转、缩放都可写成同一种“左乘 3×3 矩阵”形式，因而可以自由地通过矩阵乘法做复合。

（二）三种基本变换矩阵

- 平移 $T(t_x, t_y)$:

$$T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = T \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}.$$

- 旋转（绕原点） $R(\theta)$ ， θ 取逆时针为正:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

- 缩放 $S(s_x, s_y)$:

$$S(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

（三）变换的复合

由于矩阵乘法满足结合律，多个变换可预先复合为一个矩阵 M ，再统一作用到所有顶点，避免对每个点重复计算多次。例如本作品中鸭子的复合变换：

$$M = T(t_x, t_y) \cdot R(\theta) \cdot S(s).$$

读法从右向左：先在原点处把鸭子缩放 s 倍 $\rightarrow$ 再绕原点旋转 $\theta \rightarrow$ 最后平移到世界位

置 (t_x, t_y) 。这恰好实现了”以鸭子身体中心为枢轴的缩放与旋转，并平移到任意位置”的效果——前提是鸭子的所有部件在局部坐标系中已经以身体中心为原点。

（四）绕任意点旋转

如果要绕世界中一个非原点的枢轴 (c_x, c_y) 旋转 θ ，公式为三个矩阵的复合：

$$R_{\text{around}}(\theta, c_x, c_y) = T(c_x, c_y) \cdot R(\theta) \cdot T(-c_x, -c_y).$$

含义:把枢轴平移到原点 → 在原点旋转 → 再平移回去。本作品的 `transforms.rotate_around` 函数即此实现。

（五）矩阵实现

```
1 def translate(tx, ty):
2     return [[1.0, 0.0, tx],
3             [0.0, 1.0, ty],
4             [0.0, 0.0, 1.0]]
5
6 def rotate(theta):
7     c, s = math.cos(theta), math.sin(theta)
8     return [[c, -s, 0.0],
9             [s, c, 0.0],
10            [0.0, 0.0, 1.0]]
11
12 def scale(sx, sy=None):
13     if sy is None: sy = sx
14     return [[sx, 0.0, 0.0],
15            [0.0, sy, 0.0],
16            [0.0, 0.0, 1.0]]
17
18 def multiply(A, B):
19     return [[sum(A[i][k] * B[k][j] for k in range(3)) for j in range(3)]
20             for i in range(3)]
21
22 def compose(*mats):
```

```

23     """左到右连乘: compose(M1, M2, M3) = M1 • M2 • M3"""
24     result = identity()
25     for M in mats:
26         result = multiply(result, M)
27     return result
28
29 def apply_point(M, x, y):
30     """[x, y, 1]^T 经过 M 变换后得到新坐标"""
31     nx = M[0][0]*x + M[0][1]*y + M[0][2]
32     ny = M[1][0]*x + M[1][1]*y + M[1][2]
33     return nx, ny

```

Listing 7: 齐次坐标变换矩阵（transforms.py 摘录）

（六）圆形部件在变换下的处理

鸭子的头、眼、瞳孔、腮红都是圆。在平移 + 旋转 + 均匀缩放的组合变换下，圆仍然是圆（不会变成椭圆）：

- 圆心 (c_x, c_y) 像普通点一样经矩阵变换；
- 半径乘以变换矩阵的均匀尺度因子 $s = \sqrt{M_{00}^2 + M_{10}^2}$ （这是 M 第一列的模——对纯 $T \cdot R \cdot S$ 而言等于缩放因子）。

故圆形部件不需要顶点离散化即可直接复用题目一已实现的中点画圆算法，效率比把圆离散成多边形再做扫描线填充更高。

4.2.2 主要功能模块设计

题目二在题目一的算法层之上增加两个新模块：

- transforms.py ——纯函数库，提供 translate / rotate / scale 三种基本变换矩阵的构造、multiply / compose 矩阵乘法、apply_point / apply_points 点变换，以及 rotate_around 等便利复合。不依赖任何渲染库，可作为通用模块复用于题目三与题目四。
- duck.py ——鸭子模型。Duck 类把每个部件的几何定义在局部坐标系中（多边形：顶点列表；圆：中心 + 半径），render(surface, M, put_fn) 方法接收一个变换矩阵 M ，把所有部件经 M 变换后写入像素。

algorithms.py 与 widgets.py 直接复用题目一的实现，**零修改**。main.py 中新增 DuckScene 与 App 两个类，分别管理”画布场景 + 变换状态”和”窗口 + UI + 事件循环”。

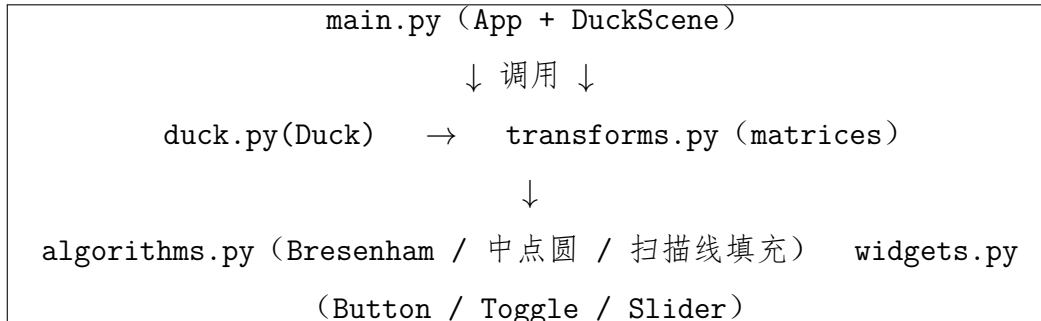


Figure 3: 题目二模块依赖关系

4.2.3 实现流程

鸭子部件定义 全部以身体中心为局部坐标原点，身体椭圆离散为 42 顶点多边形，腹部 32 顶点，翅膀 11 顶点曲线多边形，尾巴 5 顶点，头 / 眼 / 瞳孔 / 腮红用圆心 + 半径表示。

```

1 class Duck:
2     def __init__(self):
3         self.body = ellipse_polygon(0, 0, 95, 58, n=42)
4         self.belly = ellipse_polygon(-8, 20, 75, 32, n=32)
5         self.wing = [(-38,-28),(-18,-42),(8,-42),(28,-32),(42,-18),
6                     (40,2),(26,14),(4,20),(-20,16),(-36,4),(-42,-12)]
7         self.tail = [(-82,-8),(-110,-38),(-95,-35),(-88,-22),(-78,-18)]
8         self.head_c, self.head_r = (72, -58), 35
9         self.beak_upper = [(98,-68),(138,-58),(98,-55),(90,-60)]
10        ...

```

Listing 8: 鸭子局部模型 (duck.py 摘录)

统一渲染入口 接收 M ，对每个部件分别走”多边形分支”或”圆分支”：

```

1 def render(self, surface, M, put_fn):
2     s = uniform_scale_of(M) # 从 M 提取均匀尺度
3

```

```

4  def tx_poly(verts):
5      return [(int(round(x)), int(round(y)))
6              for x, y in apply_points(M, verts)]
7
8  def tx_circle(c, r):
9      cx, cy = apply_point(M, *c)
10     return int(round(cx)), int(round(cy)), max(1, int(round(r * s)))
11
12     # 后→前 渲染 (10 个部件依次叠加)
13     fill_poly(tx_poly(self.tail), COLOR_WING)
14     fill_poly(tx_poly(self.body), COLOR_BODY)
15     fill_poly(tx_poly(self.belly), COLOR_BELLY)
16     outline_poly(tx_poly(self.body), COLOR_OUTLINE)
17     fill_poly(tx_poly(self.wing), COLOR_WING)
18     ...
19     cx, cy, r = tx_circle(self.head_c, self.head_r)
20     fill_circle(cx, cy, r, COLOR_HEAD)
21     ...

```

Listing 9: 鸭子统一渲染 (duck.py 摘录)

复合变换的组装 主程序根据当前状态变量（旋转角、缩放、平移）组装变换矩阵。注意复合顺序“ $T \cdot R \cdot S$ ”读为先缩放后旋转再平移：

```

1  def transform_matrix(self):
2      """M = T(画布中心 + 偏移) · R(角度) · S(缩放)"""
3      return compose(
4          translate(CANVAS_CX + self.offset_x, CANVAS_CY + self.offset_y),
5          rotate(math.radians(self.angle_deg)),
6          scale(self.scale_factor),
7      )

```

Listing 10: 复合变换组装 (main.py 摘录)

圆周动画——演示矩阵复合 让鸭子沿圆周运动，同时朝向切线方向，这恰好用到 $T \cdot R$ 的复合：

```
1 def _step_animation(self):
2     if not self.animate_circle: return
3     self.anim_t += 0.012
4     R = 180
5     self.offset_x = math.cos(self.anim_t) * R # 圆周上的平移分量
6     self.offset_y = math.sin(self.anim_t) * R
7     # 让鸭子始终面向运动切线方向
8     self.angle_deg = math.degrees(self.anim_t) + 90.0
```

Listing 11: 圆周动画（main.py 摘录）

鼠标拖拽 在画布范围内按下左键后记录起始鼠标位与起始偏移，拖动时按相对位移更新平移分量，并同步滑块显示：

```
1 if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
2     if 落在画布内 and 不在控件上 and not 动画进行中:
3         self.dragging = True
4         self.drag_start_mouse = event.pos
5         self.drag_start_offset = (self.scene.offset_x, self.scene.offset_y)
6 elif event.type == pygame.MOUSEMOTION and self.dragging:
7     dx = event.pos[0] - self.drag_start_mouse[0]
8     dy = event.pos[1] - self.drag_start_mouse[1]
9     self.scene.offset_x = self.drag_start_offset[0] + dx
10    self.scene.offset_y = self.drag_start_offset[1] + dy
11    self._sync_sliders_from_scene() # 滑块跟随
```

Listing 12: 拖拽平移（main.py 摘录）

本地坐标轴可视化 把局部坐标系的 (0,0)、(80,0)、(0,80) 三点都经 M 变换后画成两条带箭头的轴，红色为 X 轴、绿色为 Y 轴。这样在旋转 / 缩放时，能直观看到鸭子的本地坐标系跟着偏转。

4.2.4 运行效果

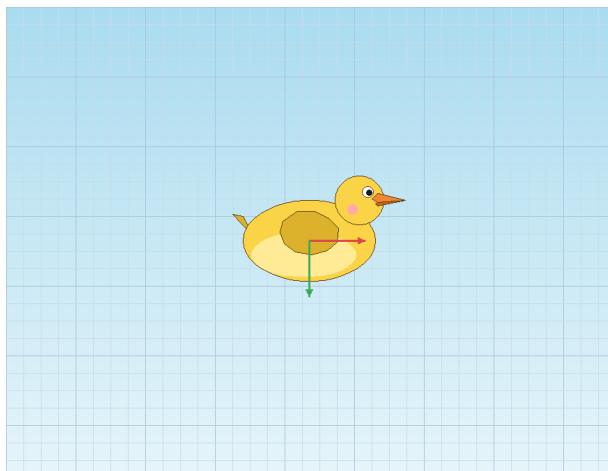


Figure 4: 题目二 • 默认姿态

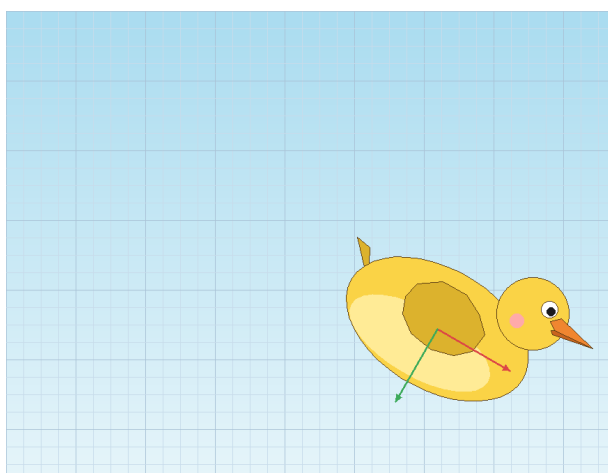


Figure 5: 旋转 30° / 缩放 1.5×/平移的鸭子姿态

4.3 题目三：游泳的鸭子

题目三是任务书指定的提高题。在题目二复用的算法栈（Bresenham 直线、中点画圆、扫描线填充、2D 齐次变换、鸭子模型）之上，本模块新引入了任务书重点提及的“图形裁剪”——具体实现 **Sutherland-Hodgman** 多边形裁剪算法，并把它落地到一个有说服力的可视场景：游泳的鸭子在水线处被切成水上水下两段，水下水下多边形用蓝色调重新填充。

4.3.1 算法原理

（一）Sutherland-Hodgman 多边形裁剪

问题：给定一个主多边形 P 和一条裁剪有向直线 L ，求 P 落在 L 内侧的那部分子多边形。

思想：依次遍历 P 的每条边 ($\text{prev} \rightarrow \text{cur}$)。约定”内侧”为 L 的右侧（叉积 ≤ 0 一侧），将边的两端点对 L 做”内 / 外”判定，得到四种情况：

prev	cur	输出到结果多边形
内	内	输出 cur
内	外	输出 (prev, cur) 与 L 的交点
外	内	先输出交点，再输出 cur
外	外	都不输出

Table 2: Sutherland–Hodgman 算法的四种边端点情况

完成一轮遍历后即得到对单条裁剪线裁剪后的子多边形。对凸的裁剪窗口（如矩形 / 三角形 / 任意凸多边形），只需对每条裁剪边逐次迭代调用上述过程，前一次的输出作为后一次的输入，最终得到完整的裁剪结果。

内 / 外判定：用叉积。点 $P = (x, y)$ 对有向直线 $L : P_1(x_1, y_1) \rightarrow P_2(x_2, y_2)$ 的有向距离指示量

$$\text{side}(P) = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1).$$

$\text{side}(P) \leq 0$ 表示 P 在 L 右侧（内侧），否则在外侧。

交点求解：直线 P_1P_2 与线段 AB 的交点用参数法求解。设交点 $= P_1 + t(P_2 - P_1)$ ，联立方程：

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}.$$

```

1 def clip_polygon_against_line(polygon, p1, p2):
2     if not polygon: return []
3     x1, y1 = p1
4     x2, y2 = p2
5
6     def side(p):
7         return (x2 - x1) * (p[1] - y1) - (y2 - y1) * (p[0] - x1)
8
9     def intersect(a, b):

```

```

10     x3, y3 = a; x4, y4 = b
11     denom = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)
12     if abs(denom) < 1e-9: return a
13     t = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / denom
14     return (x1 + t * (x2 - x1), y1 + t * (y2 - y1))
15
16 result = []
17 n = len(polygon)
18 for i in range(n):
19     cur, prev = polygon[i], polygon[i - 1]
20     cur_in = side(cur) <= 0
21     prev_in = side(prev) <= 0
22     if cur_in:
23         if not prev_in: result.append(intersect(prev, cur))
24         result.append(cur)
25     elif prev_in:
26         result.append(intersect(prev, cur))
27 return result

```

Listing 13: Sutherland–Hodgman 裁剪 (algorithms.py 摘录)

针对常用的水平线裁剪，再封装一个便利函数：

```

1 def clip_polygon_horizontal(polygon, y_line, keep="below"):
2     # pygame y 向下：方向 +x 时叉积 <= 0 一侧 = 水线上方
3     if keep == "below":
4         p1, p2 = (1, y_line), (0, y_line)
5     else:
6         p1, p2 = (0, y_line), (1, y_line)
7     return clip_polygon_against_line(polygon, p1, p2)

```

Listing 14: 水平线裁剪封装

（二）游泳动画的物理直觉

游泳动画由复合变换 $M = T \cdot R \cdot S$ 的三个分量分别承载三种运动：

- 水平游动 $\Rightarrow T$ 的 t_x ：每帧 $t_x += v \cdot d$ ，其中 v 是游泳速度、 $d \in \{+1, -1\}$ 是方向；

碰到画布边缘 d 自动取反并把鸭子缩放因子 $s_x = \pm 1$ 翻转，保证头朝运动方向。

- 上下浮沉 $\Rightarrow T$ 的 t_y : $t_y = y_{\text{水线}} - 28 + A_b \sin(\omega t)$, A_b 为浮沉幅度。
- 左右摇摆 $\Rightarrow R$ 的旋转角: $\theta = A_r \sin(1.3 \omega t)$ 。频率略高于浮沉，使两种运动节奏交错，看起来更自然。

（三）水面正弦扰动

水线 $y = y_w$ 用一维正弦扰动表现波动：

$$y_{\text{波面}}(x) = y_w + A_w \sin(kx + \omega_w t),$$

其中 $A_w \approx 2.5$ 像素、 $k = 0.025$ 频率系数。每帧沿 x 方向逐像素绘制白色细线，并在其上方再画一像素更亮的颜色，营造水面”细高光”的视觉效果。

（四）涟漪

涟漪是时变粒子系统。每个涟漪是一个三元组 (c_x, c_y, r, t) ：圆心固定，半径 r 每帧增长 ~ 1.4 像素，年龄 t 计数。用中点画圆算法生成圆周像素，但只保留位于水面线附近 $(|y - y_w| \leq 2)$ 的部分，模拟”水面波纹”而非完整圆。颜色按生命比例线性渐隐：

$$\text{color}(t) = \text{base} \cdot \left(1 - \frac{t}{t_{\max}}\right).$$

速度越大涟漪发射频率越高（冷却时间 = $\max(8, 28 - 8v)$ 帧）。

4.3.2 主要功能模块设计

题目三完全复用题目二的 `transforms.py` / `duck.py` / `widgets.py, algorithms.py` 在原有四种基本算法之外新增两个裁剪函数。`main.py` 中实现 `SwimmingScene` 与 `App` 两个类。

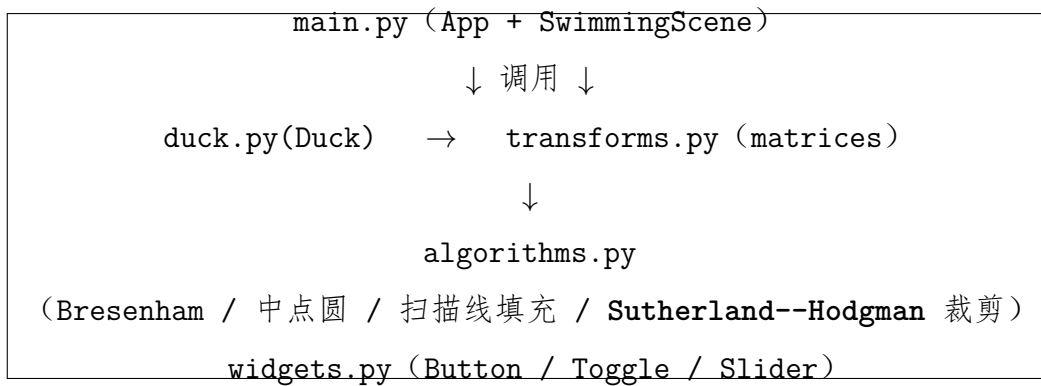


Figure 6: 题目三模块依赖关系（粗体为新增算法）

SwimmingScene 的关键状态：

- duck_x, swim_dir, swim_speed —— 鸭子水平位置、方向、速度；
- bob_amp, rock_amp_deg —— 浮沉与摇摆幅度；
- water_y_ratio —— 水线高度（占画布高度比例）；
- ripples —— 涟漪粒子列表 [{cx, cy, r, age, max_age}, ...]；
- theme_name —— 三套配色之一（白昼 / 黄昏 / 月夜）；
- show_clipping / show_clip_debug / show_ripples —— 视图开关。

4.3.3 实现流程

游泳变换矩阵的组装 三种运动通过 $T \cdot R \cdot S$ 一次性叠加：

```

1 def _duck_transform(self):
2     t = self.frame * 0.05
3     bob_y = self.bob_amp * math.sin(t)
4     rock = math.radians(self.rock_amp_deg) * math.sin(t * 1.3)
5     sx = -1.0 if self.swim_dir < 0 else 1.0 # 头朝运动方向
6     wy = self.water_y
7     cy = wy - 28 + bob_y # 略高于水面，腹部入水
8     return compose(
9         translate(self.duck_x, cy), # 水平游动 + 浮沉
10        rotate(rock), # 左右摇摆
11        scale(sx, 1.0), # 镜像翻转
12    )

```

Listing 15: 游泳变换组装 (main.py 摘录)

水下染色——裁剪算法的实战应用 鸭子先按平时的方式完整渲染一次；然后取出身体和腹部多边形，把世界坐标顶点对水线做 Sutherland–Hodgman 裁剪，得到水下手多边形，再用扫描线填充以水蓝色覆盖在已渲染的鸭子之上：

```
1 def _draw_duck(self):
2     M = self._duck_transform()
3     # 1) 整鸭按常规渲染
4     self.duck.render(self.surface, M, self._put)
5
6     # 2) 把身体/腹部多边形裁剪到水线下方，重新填充水色
7     if self.show_clipping:
8         wy = self.water_y
9         uw_color = self.theme["underwater"]
10        for part in (self.duck.body, self.duck.belly):
11            world_verts = [(int(round(x)), int(round(y)))
12                           for x, y in apply_points(M, part)]
13            below = clip_polygon_horizontal(world_verts, wy, keep="below")
14            below = [(int(round(x)), int(round(y))) for x, y in below]
15            if len(below) >= 3:
16                self._put(self.surface,
17                           scanline_fill_polygon(below), uw_color)
```

Listing 16: 水下染色 (main.py 摘录)

涟漪的发射、更新与渲染

```
1 def _emit_ripple(self):
2     bx = int(self.duck_x - 80 * self.swim_dir)
3     self.ripples.append({"cx": bx, "cy": self.water_y, "r": 2.0,
4                          "age": 0, "max_age": 70})
5
6 def _update_ripples(self):
```

```

7     for rp in self.ripples:
8         rp["r"] += 1.4
9         rp["age"] += 1
10    self.ripples = [r for r in self.ripples if r["age"] < r["max_age"]]
11
12    def _draw_ripples(self):
13        wy = self.water_y
14        base = self.theme["ripple"]
15        for rp in self.ripples:
16            alpha = 1.0 - rp["age"] / rp["max_age"]
17            c = tuple(int(b * alpha) for b in base)
18            pts = midpoint_circle(rp["cx"], rp["cy"], int(rp["r"]))
19            for x, y in pts:
20                if abs(y - wy) <= 2: # 只保留水面附近的圆弧
21                    if 0 <= x < self.w and 0 <= y < self.h:
22                        self.surface.set_at((x, y), c)

```

Listing 17: 涟漪粒子系统（main.py 摘录）

自动游泳与边界反弹

```

1    def _step(self):
2        if self.paused or self.dragging:
3            return
4        self.frame += 1
5        self.duck_x += self.swim_speed * self.swim_dir
6        margin = 100
7        if self.duck_x < margin:
8            self.duck_x = margin; self.swim_dir = 1
9        elif self.duck_x > self.w - margin:
10            self.duck_x = self.w - margin; self.swim_dir = -1
11        self.ripple_cd -= 1
12        if self.ripple_cd <= 0:
13            self._emit_ripple()
14            self.ripple_cd = max(8, int(28 - self.swim_speed * 8))

```

```
self._update_ripples()
```

Listing 18: 步进逻辑（main.py 摘录）

裁剪调试可视化 开启“裁剪调试”开关后：

- 在画布上叠加水线（红色横线）；
- 给裁剪后的水分子多边形描黄色边框（Bresenham 直线）；
- 在 $|y - y_w| \leq 1$ 的裁剪交点位置画一个红色小圆点。

这让 Sutherland-Hodgman 算法的输出可以直接被肉眼验证——能看到交点正好落在水线上、子多边形与水线无缝相接。

4.3.4 运行效果

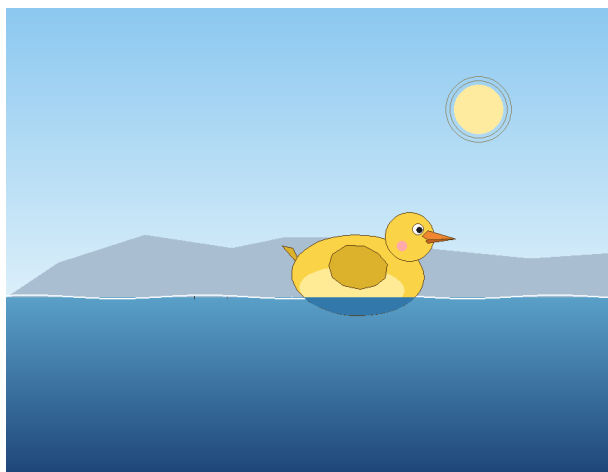


Figure 7: 题目三 • 游泳的鸭子（白昼）

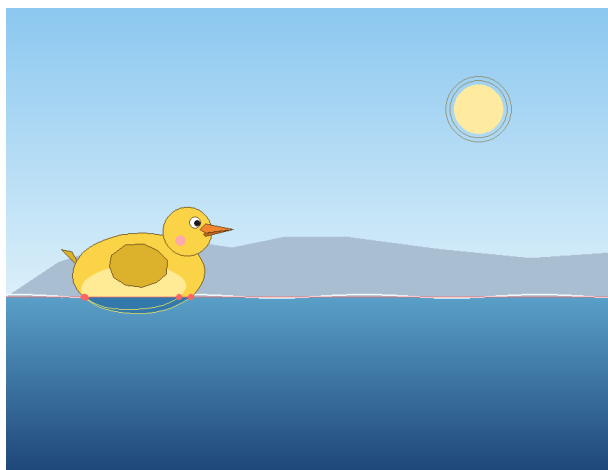


Figure 8: Sutherland-Hodgman 裁剪可视化（调试模式）

4.4 题目四：松树与松果

题目四是任务书指定的难题。本模块是整个课程设计的”算法集大成者”——直接复用前三题的算法库，新引入 **De Casteljau Bezier 曲线** 作为关键图形学算法，并以基于矩阵的层级化变换与 简单刚体物理两套机制驱动整个场景的动画。

4.4.1 算法原理

（一）De Casteljau 算法与 Bezier 曲线

n 阶 Bezier 曲线由 $n + 1$ 个控制点 $P_0, P_1, \dots, P_n$ 定义，参数形式为

$$B(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i P_i, \quad t \in [0, 1].$$

直接代入二项展开计算数值稳定性差，且每阶需要重新推导。**De Casteljau 算法**给出了一种递推式求值方法，只用反复的线性插值：

$$\begin{aligned} P_i^{(0)} &= P_i, \quad i = 0, 1, \dots, n, \\ P_i^{(k)} &= (1-t) P_i^{(k-1)} + t P_{i+1}^{(k-1)}, \quad k = 1, \dots, n, \end{aligned}$$

最终 $P_0^{(n)}$ 即为曲线上参数 t 处的点。每一轮把控制点数减少 1，经过 n 轮（ $O(n^2)$ 次插值）即可得到结果。该算法数值稳定、几何直观（每轮的中间点连成的折线逐渐”塌缩”到曲线上一点），并可推广到任意阶。

```

1 def de_casteljau(control_points, t):
2     pts = [(float(x), float(y)) for x, y in control_points]
3     while len(pts) > 1:
4         nxt = []
5         for i in range(len(pts) - 1):
6             x = (1 - t) * pts[i][0] + t * pts[i + 1][0]
7             y = (1 - t) * pts[i][1] + t * pts[i + 1][1]
8             nxt.append((x, y))
9         pts = nxt
10    return pts[0]
11
12 def bezier_curve(control_points, segments=24):
13    return [de_casteljau(control_points, i / segments) for i in range(segments
        + 1)]

```

Listing 19: De Casteljau 算法 (algorithms.py 摘录)

(二) Bezier 曲线在松树枝叶中的应用

每层 tier 的下垂底边用一条二次 Bezier 生成：控制点取

$$P_0 = (-w, y_{\text{底}}), \quad P_1 = (0, y_{\text{底}} + 0.4h), \quad P_2 = (w, y_{\text{底}}),$$

其中 w 为该层半宽、 h 为该层高度。 P_1 位于两端点下方 (pygame y 向下, $y_{\text{底}} + 0.4h$ 比端点更靠下), 故曲线向下凸, 给枝叶一种自然垂坠感。采样 18 段后与左右斜边及顶点拼成 20 顶点多边形, 直接送入扫描线填充。

(三) 基于矩阵的层级化摇摆

设树根在世界位置 (x_0, y_0) , 则树 $\rightarrow$ 世界的基变换为 $M_{\text{base}} = T(x_0, y_0)$ 。每层 tier 在树坐标系中以自身底边 $(0, y_i)$ 为支点旋转, 旋转矩阵为

$$M_{\text{sway}, i} = T(0, y_i) \cdot R(\theta_i) \cdot T(0, -y_i),$$

其中摆动角度按高度递增：

$$\theta_i = A_i \cdot w_{\text{wind}} \cdot \sin(\omega t + \varphi_i), \quad A_i = A_{\min} + (A_{\max} - A_{\min}) \cdot \frac{i}{N-1}.$$

最终每层的世界变换由树基变换与自身摇摆复合：

$$M_{\text{world},i} = M_{\text{base}} \cdot M_{\text{sway},i}.$$

这与角色骨骼动画中”父骨头矩阵 · 子骨头局部矩阵”的原理一致，是**层级化变换**（hierarchical transformation）的最简实现。

枝头位置——也即松果挂载点——同样位于 tier 局部坐标中（底边的 4 个采样点之一），经 $M_{\text{world},i}$ 变换即得每帧的世界位置，让挂着的松果自然随着枝叶一起摇晃。

（四）松果的物理仿真

每颗松果维护一个三状态机：

状态	位置来源	转换条件
ATTACHED	每帧同步对应枝头世界位置	摇晃按钮 / 自动倒计时 / 鼠标点击附近
FALLING	物理积分（重力 + 风 + 阻力）	$y \geq y_{\text{地面}}$ 时落地
LANDED	停在落地位置	无（直至重置）

Table 3: 松果状态机

物理积分（每帧 $\Delta t = 1$ ）：

$$v_y += g, \quad v_x += w_{\text{wind}} \cdot k_w, \quad \begin{matrix} v_x *= d \\ v_y *= d \end{matrix}, \quad x += v_x, \quad y += v_y, \quad \theta += v_a,$$

其中 $g = 0.35$ 为重力加速度、 $k_w = 0.05$ 为风力耦合系数、 $d = 0.992$ 为空气阻力。脱落瞬间给定一个小的随机初速 (v_x, v_y, v_a) ，模拟松果”被颠下来”的不规则运动。

4.4.2 主要功能模块设计

题目四进一步丰富了通用算法库，并新增两个领域模块：

- `algorithms.py`——在原有四种算法基础上新增 `de_casteljau` 与 `bezier_curve` 两个函数。
- `tree.py`——新增。`TreeTier` 封装一层枝叶（Bezier 底边 + 摇摆矩阵），`PineTree` 管理树干与多层 tier，提供 `render` 与 `branch_tip_world_positions` 接口。
- `pinecone.py`——新增。`Pinecone` 实现三状态机、物理积分和外观渲染（橄榄形多边形 + 6 条 Bresenham 鳞片纹理）。
- `transforms.py` / `widgets.py`——直接复用，零修改。
- `main.py`——`ForestScene`（场景管理、参数控制、主题、松果调度）+ `App`（窗口 + UI + 事件循环）。

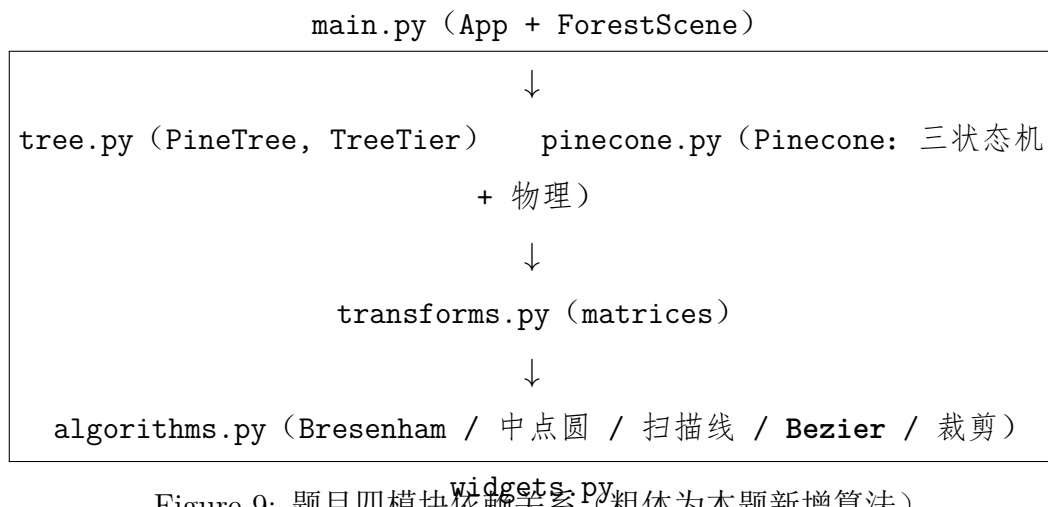


Figure 9: 题目四模块依赖关系（粗体为本题新增算法）

4.4.3 实现流程

Bezier 曲线生成枝叶轮廓

```
1 class TreeTier:
2     def __init__(self, base_y, half_width, height, ...):
3         self.base_y, self.w, self.h = base_y, half_width, height
4         # 底边: 二次 Bezier, 控制点 P1 在 (0, base_y + 0.4h)
5         bottom = bezier_curve(
6             [(-self.w, self.base_y),
```

```

7         ( 0.0, self.base_y + 0.4 * self.h),
8         ( self.w, self.base_y)],
9     segments=18,
10 )
11 apex = (0.0, self.base_y - self.h)
12 self.local_polygon = bottom + [apex] # 19 + 1 = 20 顶点
13 # 枝头: 从底边采样里挑 4 个点作为松果挂载位置
14 n = len(bottom)
15 self.branch_tips_local = [bottom[i] for i in
16                             (int(n*0.15), int(n*0.4),
17                             int(n*0.6), int(n*0.85))]

```

Listing 20: tier 静态轮廓 (tree.py 摘录)

tier 摇摆矩阵与世界变换

```

1 def tier_world_matrices(self, base_M, t, wind_strength=1.0):
2     """[(tier, M_world), ...] 每层叠加自身摇摆 + 父变换"""
3     result = []
4     for tier in self.tiers:
5         theta = (math.radians(tier.sway_amp_deg) * wind_strength
6                 * math.sin(t + tier.sway_phase))
7         M_sway = compose(translate(0, tier.base_y),
8                           rotate(theta),
9                           translate(0, -tier.base_y))
10        result.append((tier, compose(base_M, M_sway))) # 父 子
11    return result

```

Listing 21: 层级化变换 (tree.py 摘录)

松果挂载位置同步 每帧 ForestScene 都要把所有 ATTACHED 状态的松果的世界位置同步到当前枝头位置 (枝头本身因摇摆而移动):

```

1 tips = self.tree.branch_tip_world_positions(base_M, t, wind_strength=1.0)
2 tips_map = {(ti, ki): pos for ti, ki, pos in tips}
3 for pc in self.pinecones:

```

```

4     if pc.state == Pinecone.STATE_ATTACHED:
5         pos = tips_map.get((pc.tier_idx, pc.tip_idx))
6         if pos is not None:
7             pc.attach_to(pos)
8     else:
9         pc.step(self.wind, self.ground_y - 2)

```

Listing 22: 松果挂载同步 (main.py 摘录)

松果物理积分

```

1 def step(self, wind_x, ground_y):
2     if self.state != self.STATE_FALLING:
3         return
4     self.vy += GRAVITY # 重力
5     self.vx += wind_x * WIND_COEF # 风力
6     self.vx *= DRAG # 空气阻力
7     self.vy *= DRAG
8     self.x += self.vx
9     self.y += self.vy
10    self.angle += self.va # 自旋
11    if self.y >= ground_y: # 落地
12        self.y = ground_y
13        self.state = self.STATE_LANDED
14        self.vx = self.vy = self.va = 0.0

```

Listing 23: 物理 step (pinecone.py 摘录)

Bezier 控制点的可视化 开启“Bezier 控制点显示”开关后，底层 tier 的 3 个控制点（红色实心圆）和它们的控制多边形（粉色折线）被画在画布上。控制点会跟随该层 tier 的摇摆一起旋转，让”曲线的几何意义”动起来：观察者可以直观感受”控制多边形向曲线靠近”这一 De Casteljau 的核心几何性质。

4.4.4 运行效果



Figure 10: 题目四 • 默认状态（春日）

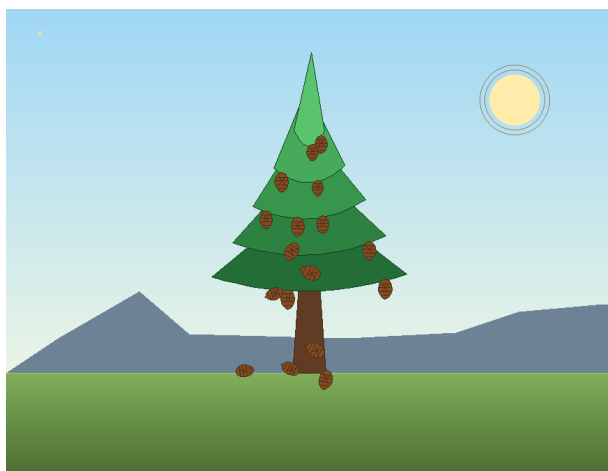


Figure 11: 物理仿真：风力 + 重力 + 自旋的松果掉落



Figure 12: De Casteljau Bezier 控制点的几何可视化

五、总结

5.1 程序结构优缺点分析

优点

- **算法与渲染解耦**: `algorithms.py` 是纯函数库, 输入几何参数、输出像素坐标, 不依赖 `pygame.Surface` 或任何渲染上下文。这样既便于单元测试, 也允许该模块直接在题目二/三/四中复用, 无需任何改动。 7×10^5 降到约 1.5×10^4 , 使得在普通笔记本上也能流畅运行。
- **交互完整**: 菜单栏 (按钮 + 开关)、参数面板 (滑块)、快捷键、保存图片、关于弹窗一应俱全, 满足任务书“界面美观、衔接自然、注重交互性”的要求。

缺点与改进方向

- **扫描线填充未做边表加速**: 教科书做法是活动边表 (AET), 扫描线推进时增量维护边表, 复杂度从 $O(\text{边数} \times \text{扫描线数})$ 降到 $O(\text{像素数})$ 。本实现采用最朴素的“逐扫描线遍历所有边”, 凹多边形顶点不多时差异不显著, 但对复杂图形仍有优化空间。

5.2 出错原因分析

开发过程中遇到的主要问题及解决:

- (1) **Pygame 字体初始化崩溃** 第一次启动时报错 `TypeError: expected str, bytes or os.PathLike object, not int`, 来自 `pygame.font.sysfont.initsysfonts_win32`。原因是本机 Windows 字体注册表中存在非字符串类型的项 (某些被损坏的字体条目), `splitext` 期望字符串而拿到整数。**解决**: 彻底跳过 `pygame.font.SysFont` 与 `pygame.font.get_fonts`, 改为通过 `pygame.font.Font(r"C:\Windows\Fonts\msyh.ttc", size)` 按文件路径加载字体, 绕开有问题的系统字体扫描过程。
- (2) **扫描线在顶点处的“漏填”或“过填”** 最初版本在多边形顶点处偶尔出现单像素瑕疵。原因是当扫描线 y 恰好等于某个顶点 y_0 时, 若该顶点被两条边同时计入, 会得到奇数个交点, 配对失败。**解决**: 约定每条边包含下端点而不含上端点 (条件 $y_0 \leq y < y_1$), 顶点只会被位于其下方的那条边计入一次。

- (3) **中文字符显示成方块** 最初使用 `pygame.font.Font(None, ...)` 加载默认字体, 所有中文显示为方块 (tofu)。**解决:** 定位到 `msyh.ttc / simhei.ttf` 等系统中文字体文件后按路径加载。
- (4) **逐帧重画整个场景导致掉帧** 初版每帧都重画渐变背景 (720 行扫描) + 山脉填充 (~50000 像素) + 月亮填充 (~8000 像素), 帧率约 12FPS。**解决:** 把这三者画到一张缓存 Surface 上, 每帧只 `blit` 复用, 仅动态元素重画, 稳定 60FPS。
- (5) **彗星拖尾不够”软”** 最初拖尾使用等亮度直线, 视觉突兀。**解决:** 沿尾巴方向按距离平方衰减 $\alpha = (1 - i/L)^{1.5}$, 让尾巴远端自然过渡到背景。
- (6) **Sutherland-Hodgman 裁剪方向反了** 题目三实现 `clip_polygon_horizontal(poly, y, keep="below")` 后单元测试发现: 用三角形 (0,0)-(10,0)-(5,10)、 $y_{\text{线}} = 5$ 时, `keep="below"` 竟然返回了上半 (含 (0,0) 与 (10,0))。原因: 算法约定”内侧”为有向直线右侧 (叉积 ≤ 0), 而 pygame 中 y 向下——一条沿 $+x$ 方向的水平线, 其右侧 (叉积 ≤ 0) 对应 $y \leq y_{\text{线}}$, 正是水线上方。**解决:** 把 `keep="below"` 对应的有向直线取反方向 ($p_1, p_2 = (1, y), (0, y)$), 让 $-x$ 方向直线的右侧 (叉积 ≤ 0) 对应 $y \geq y_{\text{线}}$, 与”水下”含义一致。教训: 坐标系朝向 (y -up vs. y -down) 必须事先核对, 几何算法的方向约定要用单元测试单独验证, 不能依赖肉眼调 UI。
- (7) **涟漪用完整圆显得不自然** 最初把涟漪当作正圆全部画出来, 俯视感太强, 与侧面游泳视角冲突。**解决:** 仍用中点画圆生成全圆像素, 但渲染时只保留满足 $|y - y_w| \leq 2$ 的部分, 得到”水面带状波纹”效果, 符合人在水边平视所看到的水波形态。
- (8) **Bezier 采样段数的精度/性能权衡** 题目四的 tier 底边 Bezier 一开始采样 8 段, 曲线在中段呈现可见的折线感, 破坏”枝叶下垂”的圆润视觉。提到 36 段后又显著影响每帧的扫描线填充耗时 (多边形顶点更多 $\rightarrow$ 边表更长)。**解决:** 折中取 18 段, 肉眼几乎察觉不到折角, 扫描线填充的额外开销可以忽略。更一般的做法是自适应分段 (根据曲线曲率细分), 但本作品的曲线很短小, 固定段数已经足够。
- (9) **层级摇摆放大失控** 题目四最初尝试做”完整层级”——每层 tier 的旋转矩阵叠加在上一层已变换的世界矩阵之上 ($M_i = M_{i-1} \cdot M_{\text{sway}, i}$), 模拟”枝叶根部跟着上一层底部一起摇”。实际上这种连乘角度累积过快: 顶层有效摆动可达 20° 以上, 看起来更像柳枝、不像松树。**解决:** 改为”父变换 + 自身摇摆”两级结构 ($M_i = M_{\text{base}} \cdot M_{\text{sway}, i}$) ——每层独立绕自己的底边摇摆, 幅度沿高度线性增加而不再连乘, 动作克制、符合松树韧性强形象。

- (10) **“挂着松果”瞬移”感** 最初把松果当成独立物体，每帧用 `attach_to(world_xy)` 覆盖位置；遇到大风时枝头瞬时变化大，松果显得不连续。**解决：**根本原因不是同步逻辑出错，而是帧率不稳让插值断层。通过把静态背景（天空 + 远山 + 地面 + 太阳 + 月亮）缓存为一张 Surface，每帧只重绘动态部分，帧率回到稳定 60FPS，问题消失。教训：表象问题先排查帧率，再怀疑逻辑。